0.1 Introduction to Suricata

0.1.1 What is Suricata

Figure 0.1: Suricata logo

Suricata is an open-source tool developed by the Open Information Security Foundation (OISF). It can works as an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS). It monitors network traffic, identifies suspicious activities, and alerts system administrators when potential threats are detected. Suricata is a flexible and efficient tool, integrating well with modern network security setups therefore offering powerful options for advanced network security architectures.

As a multi-threading tool, Suricata excels at analyzing network traffic. It's optimized to work with modern multi-core CPUs or GPU, ensuring high performance, especially with compatible network cards and software.

0.1.2 Development History

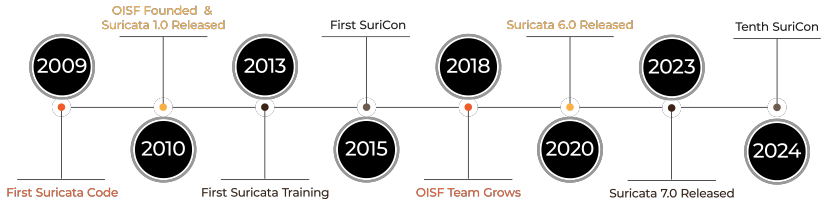


Figure 0.2: Suricata Timeline

Origins and Early Development

The development of Suricata began in 2007, initiated by Victor Julien and Matt Jonkman, with contributions from William Metcalf. This early phase involved creating the initial code and presenting a prototype in early 2008, setting the stage for future advancements. The collaborative effort during this period was crucial in establishing the technical foundation for Suricata's capabilities.

Initial Releases and Growth

The first public beta version of Suricata was released on December 31, 2009, marking a significant step toward public testing and feedback. This beta release

was announced on platforms like Slashdot, indicating early community interest. Following this, the first standard release, version 1.0, was launched in July 2010. This release cycle established Suricata as a viable tool, with major updates typically occurring every three months, reflecting a commitment to continuous improvement. For example, the first public, in-person training in 2013 at Hack.lu in Luxembourg, and the inception of SuriCon conferences starting in 2015 has make Suricata's growth included significant community engagement. Additionally, later milestones like the release of Suricata 6.0 in 2020 and Suricata 7.0 in 2023. For more detail on important milestone of Suricate, see this Suricata Timeline

0.2 Suricata Architecture

Suricata is a high-performance, open-source IDS/IPS and network security monitoring engine. It leverages native multi-threading, efficient algorithms, and modular preprocessing to achieve real-time analysis on multi-core systems without manual traffic balancing.

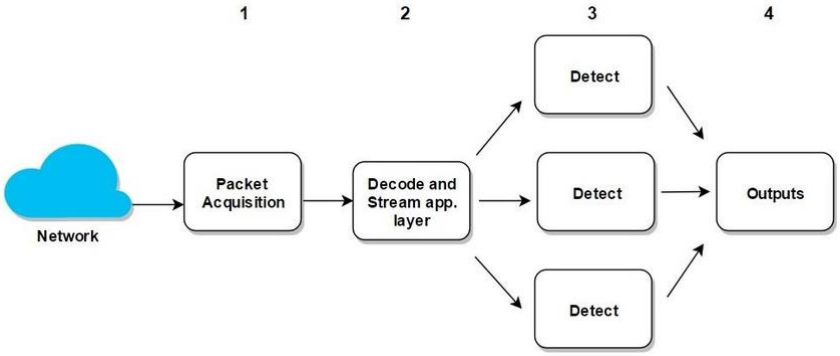
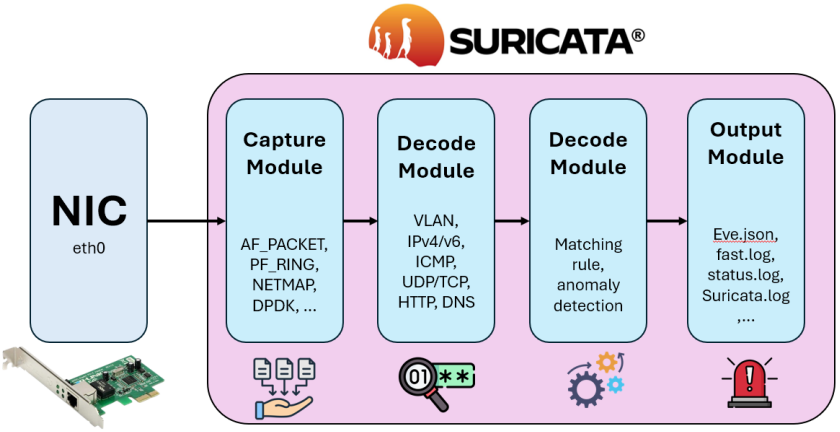


Figure 0.3: Suricata Architecture

Its core components—***packet capture***, ***decoding***, ***detection***, and ***output***—work in concert to capture, normalize, inspect, and report on network traffic. We will cover each of these components in detail.



0.2.1 Packet Capture

Purpose: The first stage is acquiring raw packets from one or more interfaces.

Method: Suricata uses different capture mechanisms depending on the platform:

Platform	Capture Mechanism	Description	Use Case
Linux	AF_PACKET (default)	Employs Linux's AF_PACKET sockets for high-speed packet capture. Supports AF_PACKET v3 for zero-copy mode. Can use eBPF for load balancing.	Inline IPS, high-performance IDS
Linux (alternative)	PF_RING	High-speed packet capture framework from ntop, with kernel bypass. Needs PF_RING kernel module and compatible NICs.	Environments needing efficient packet capture with load balancing.
Linux / FreeBSD / macOS	Netmap	Kernel bypass framework for fast packet I/O. Requires NETMAP support in the OS and NIC drivers.	Specialized high-speed packet processing tasks.
Linux (5.3+)	AF_XDP	Leverages eXpress Data Path for ultra-low latency packet processing. Requires kernel 5.3+ and compatible NIC drivers.	High-performance scenarios requiring minimal latency.
Linux	NFQUEUE	Captures packets via Netfilter queue.	Inline IPS setups: Integrates with iptables/nftables; suitable for packet modification.
Linux	DPDK	High-performance packet processing using Data Plane Development Kit, bypassing kernel for low-latency networking. Requires dedicated CPU cores and NIC support.	High-speed packet processing, low-latency applications, network function virtualization (NFV).
Windows	WinDivert	A kernel-level driver that intercepts, filters, and can inject packets in Windows.	Inline IPS, traffic redirection, deep filtering. Requires manual installation and configuration.
Generic (cross-platform)	PCAP (libpcap) (default fallback)	Standard libpcap for compatibility across systems where other methods are unavailable.	Simple setups, offline analysis. Limited performance; not ideal for high-throughput environments.

Table 1: Packet Capture Mechanisms Across Different Platforms

Packet capture can be distributed across multiple threads to handle high-speed networks.

0.2.2 Decoding

Purpose: Each packet is processed through a series of decoders that extract information from the data link layer (L2) up to the application layer (L5) in TCP/IP

model. This decoded information is stored in an internal packet representation and will be used later by the detection engine.

Method: Suricata processes packets through a modular pipeline of decoders, each tailored to a specific protocol or network layer. This ensures efficient and accurate parsing, critical for handling encapsulated protocols (e.g., Ethernet $\rightarrow$ IPv4 $\rightarrow$ TCP). For each protocol, the packet structure is updated to point to the corresponding header or payload in the packet data. Depending on the decoded information, Suricata calls the next decoder to process the next layer of the packet until no more layers remain to decode (recursive decoding).

The table below summarizes key information extracted at each layer:

Layer	**Information Extracted**
L2 – Data Link	Ethernet frame, MAC addresses, VLAN tags
L3 – Network	IP addresses, header, fragmentationand reassembly
L4 – Transport	Ports, TCP/UDP flags, sequence numbers, checksum, TCP stream reassembly
L5 – Application	Automatically identifies protocols (based on bahaviour, not just port since connection may not use standard port) and extracts metadata such as HTTP URIs, DNS query names, and TLS certificate information.

Table 2: Summary of decoding layers and extracted information

To view these plugins in Suricata, you can check the Suricata's official Github repository, under `src/decode-*`

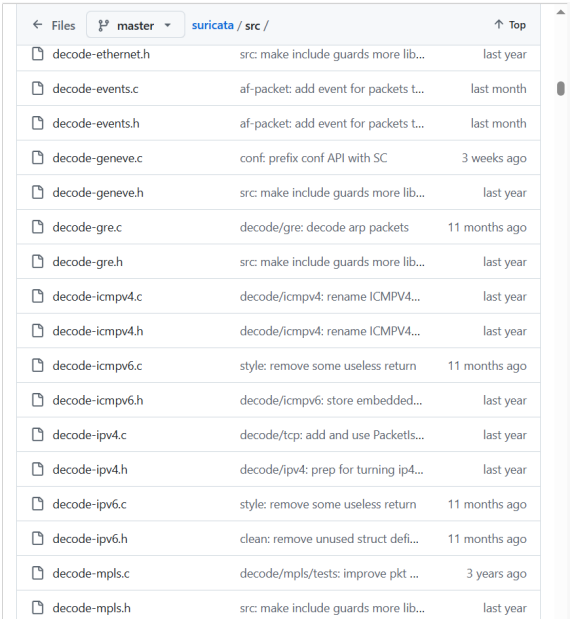


Figure 0.4: Suricata source code files for decoders

For instance:

- `decode_ipv4.c` contains functions to parse IPv4 headers and extract fields such as source IP, destination IP, protocol type, and header length.
- `decode_ethernet.c` handles Ethernet headers, extracting source and destination MAC addresses and the EtherType field.

The result of this decoding process is a rich *packet structure* that carries all relevant metadata and payload fragments to the next stage of Suricata's processing pipeline.

0.2.3 Detection

Suricata's detection engine is optimized for high-volume traffic, leveraging modern multi-core CPU architectures through native multi threading. Unlike other IDS/IPS systems requiring manual load balancing, Suricata automatically distributes workloads across threads, simplifying deployment in high-traffic environments. Packet capture, decoding, and detection are all multi-threaded, enabling horizontal scaling. Detection engine is the heart or every IDS/IPS system. This detection engine can work by signature-bases, or anomaly-based as mentioned in the first part of the report.

a, Signature-Based Detection

Purpose: Identifies threats by applying rules to decoded packets. **Method:** Using rules, compatible with Snort's format, include an action (e.g., pass, drop), header (e.g., protocol, source/destination IP, ports), and options (e.g., content match-

ing, protocol-specific keywords). Example: `alert tcp any any -> 192.168.1.0/24 111 (content:"|00 01 86 a5|"; msg:"mountd access";)`. This rule structure will be discussed in detail in the next section. Then, extracted packet information from decode stage is compared against rules using efficient algorithms like Aho-Corasick, Hyperspace, or PCRE for content matching.

b, Anomaly Detection

Anomaly-based detection is not researched within the scope of this project.

0.2.4 Output

After detection, potential threats and related packets are forwarded to the logging/alert component, configurable via `suricata.yaml`. By default, Suricata outputs `fast.log` for concise overviews and `eve.json` for detailed JSON logs, ideal for SIEM integration. In IPS mode, Suricata can block malicious traffic with inline mode or firewall integration, requiring specific setup.

Additional, as an open-source tool, Suricata's architecture supports customization. Components can be modified, plugins adding, to meet your organization specific needs.

0.3 Suricata Rule Structure

Suricata rules form the core of its detection capabilities. They specify which network traffic to inspect and what actions to take when specific patterns are matched. These rules are signature-based: each rule defines a signature, and when network traffic matches that signature, Suricata can generate alerts, log events, or even block the traffic (in IPS mode). Rules may be:

- Sourced from community-maintained sets such as Emerging Threats. These rulesets can be easily download, update and manage with `suricata-update` dependency.
- Custom-written by your own to meet specific organizational needs, making Suricata highly flexible and adaptable.

Suricata is also compatible with Snort rules, but with additionally offers extended keywords and capabilities. In this section , I cover briefly about Suricata rule structure, for detail information about this rule structure as well as rule options, please refer to the official Suricate document guide.

A typical Suricata rule consists of three main parts:

1. **Action:** Specifies what Suricata does when the rule matches.

2. **Header:** Defines the protocol, IP address, port and direction of the rule
3. **Options:** Provide detailed matching conditions and metadata.

The overall syntax is:

```
action protocol source_address source_port
    -> destination_address destination_port (options)
```

Example of a Suricata rule, this will also be a case study for some of our sections below:

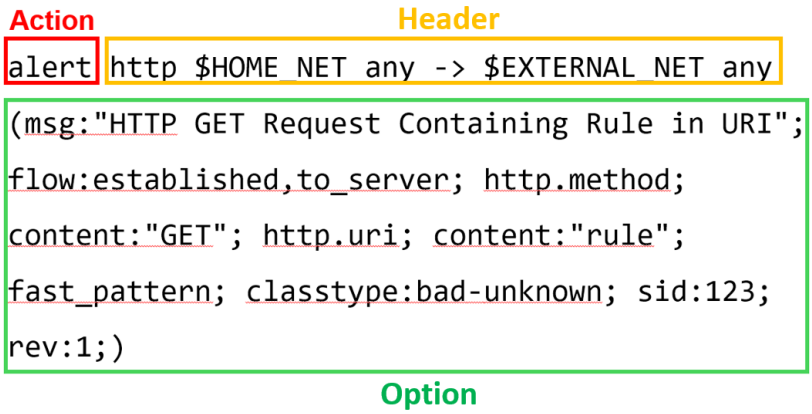


Figure 0.5: Example of a Suricata rule

In the following we explore each part in detail.

0.3.1 Actions

The `action` specifies Suricata's response when a rule matches (behavior also depend on whether Suricata is working in IDS or IPS mode). Available actions include:

Action	IDS Mode	IPS Mode
alert	Generate an alert; allow the packet.	Generate an alert; allow the packet.
pass	Stop futher inspection; allow the packet.	Stop futher inspection; allow the packet.
drop	(Treated as alert)	Drop the packet; generate an alert.
reject	(Treated as alert)	Drop the packet; send TCP RST or ICMP unreach error to the sender.
reject_src	(Treated as alert)	Drop the packet; send TCP RST or ICMP to the sender.
reject_dst	(Treated as alert)	Drop the packet; send TCP RST or ICMP to the receiver of matching package.
reject_both	(Treated as alert)	Drop the packet; send TCP RST or ICMP to both sides off the conversation.

Table 3: Behavior of Suricata actions in IDS vs. IPS modes

IN IPS mode, using any of the `reject` actions also perform `drop`.

0.3.2 Header

The header begins with the protocol, followed by source and destination IPs and ports, and the direction operator.

Figure 0.6: Example of Suricata rule header

a, Protocol

This keyword will specify which network protocol Suricata need to concerns. Suricata supports four basic protocol:

- Network layer: `ip`, `icmp`
- Transport layer: `udp`, `tcp` (and some additional TCP related options like: `tcp-pkt` for matching content in individual tcp packages, `tcp-stream` for matching content only in a reassembled tcp stream)

Upon these basic protocol, there are some Layer 5 (Application Layer) protocols are also supported: `http` (either `http1` or `http2`), `ftp`, `tls` (this includes `ssl`), `smb`, `dns`, `dcerpc`, `dhcp`, `ssh`, `smtp`, `imap`, `pop3`, `modbus*`, `dnp3 *`, `enip*`, `nfs`, `ike`, `krb5`, `bittorrent-dht`, `ntp`, `rfb`, `rdp`, `snmp`, `tftp`, `sip`, and `websocket`.

(Protocols with * are disable by default, require manual enable in the `suricata.yaml` config file.)

b, IP Addresses and Ports

IP address can be specified by single values, ranges, negations, or variables defined in the configuration.

Type	Example	Description
Single IP	`191.168.1.8`	Matches exactly with this address.
CIDR range	`192.168.0.0/24`	Matches all addresses in this /24 subnet.
Variable	`$HOME_NET`	This variable is defined inside `suricata.yaml`.
Negation	`!$HOME_NET`	Matches any address outside $HOME_NET.
Grouping	`[$HOME_NET, 192.168.1.0/24]`	Union of multiple networks.
Any	`any`	Matches all IP addresses.

Table 4: IP address specification

Suricata supports both **Ipv4** and **IPv6**

Port specifications follow the same patterns:

Type	Example	Description
Single Port	`80`	Matches exactly this port
Any Port	`any`	Matches all ports (0-65535).
Port Range	`100-102`	Matches a continuous range of ports from start to end.
Negation	`!80`	Matches any port except the specified one.
Grouping Operator	`[100:120,!101]`	All port in range 100 to 102, except 101.

Table 5: Description of port matching types and examples

c, Direction

Suricata uses three different arrow operators to control which way traffic is evaluated:

- *Unidirectional*: ”->”

  Matches only packets flowing from the source to the destination as written. For example,

  ```
  alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (...)
  ```

  will only inspect packets sent from `$EXTERNAL_NET` toward `$HOME_NET` on port 80.

- *Session-aware bidirectional*: ”=>”

  Behaves like `->` for initial packet direction, but allows matching on both sides of an established transaction when used with keywords that track session state (e.g. `flow:established;` or `flowbits`). In effect, you still write the rule in one direction, but Suricata will check both client-to-server and server-to-client payloads once the session is seen. Example:

  ```
  alert http $HOME_NET any => $EXTERNAL_NET any
  (flow:established; content:"password="; ...)
  ```

  This will catch “password=” in either direction after the HTTP session is established.

- *Fully bidirectional*: ”<>”

  Instantly inspects packets in both directions without requiring session keywords. Useful when you want a single rule to match client-to-server or server-to-client traffic equally:

  ```
  alert tcp $HOME_NET any <> $EXTERNAL_NET any
  (content:"secret"; ...)
  ```

When to use each:

-> Simple, stateless patterns where only one direction matters.

=> Stateful checks—e.g. look for data both ways once a session is set up.

<> Stateless bidirectional matching, at the cost of slightly more CPU since both

directions are always checked.

Note that there is **no** <− operator; use separate rules or variables if needed for reverse-only inspection.

0.3.3 Options

Options are enclosed in parentheses "`()`" after the header. Options contain key-value pair, seperated by semicolons "`;`" like this:

```
(option1:value1; option2:value2; ...)
```

Options may be grouped into four categories:

- **Metadata:** Metadata or additional visualize when logging/alert, no effect on inspection of the package (e.g., `msg`)
- **Payload:** Content-based matching (e.g., `content`, `pcre`).
- **Non-Payload:** Inspect condition that not related to package payload, like fields in package header, flow attributes (e.g., `flow`, `ttl`, `dsize`).
- **Post-Detection:** Actions after a match (e.g., `logto`, `tag`).

a, Metadata Options

General options provide descriptive information and references without affecting the matching logic:

1. **msg** (Message)

   Give the contextual information about the signatures and the alert. This is the message that will be displayed when the rule is triggered. It should be descriptive enough to understand what the rule is doing.

   - **Syntax:** `msg:"text_here"`, but notice that escape characters `"`, `;`, and `\` in a text string must be escaped with a backslash `\`.
   - **Example** with out case study rule:

   ```
   (msg:"HTTP GET Request Containing Rule in URI";
   flow:established,to_server; http.method;
   content:"GET"; http.uri; content:"rule";
   fast_pattern; classtype:bad-unknown; sid:123;
   rev:1;)
   ```

2. **sid** (Signature ID)

A unique identifier for the rule. Must be unique across fore all rules within the same rule group id (`gid`).

- **Syntax:** `sid:<number>`
- **Example** with out case study rule:

```
alert http $HOME_NET any -> $EXTERNAL_NET any
(msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method;
content:"GET"; http.uri; content:"rule";
fast_pattern; classtype:bad-unknown; sid:123;
rev:1;)
```

3. **rev** (Revision)

   Each sid can have multiple revisions. This is used to track changes to the rule over time. Whenever a rule is updated, the rev number should be incremented by the rule writer.

   - **Syntax:** `rev:<number>`
   - **Example** with out case study rule:

```
alert http $HOME_NET any -> $EXTERNAL_NET any
(msg:"HTTP GET Request Containing Rule in URI";
flow:established,to_server; http.method;
content:"GET"; http.uri; content:"rule";
fast_pattern; classtype:bad-unknown; sid:123;
rev:1;)
```

4. **gid** (Group ID)

   Use to organize rules into separate groups. Within the same group, each rule must have a unique Signature ID (sid). However, the same sid can appear in different groups without conflict. For example, gid:1 and gid:2 can both contain a rule with sid:1000001, and these rules are considered distinct. By default, Suricata assigns a gid of 1 to all rules. This grouping has no technical impact on detection—it is primarily used for identification and logging purposes, such as in the fast.log output.

   - **Syntax:** `gid:<number>`

- **Example** in the output of **fast.log** file:

```
07/12/2022-21:59:26.713297 [**] [1:123:1] HTTP GET Request Containing Rule in
URI [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP}
192.168.225.121:12407 -> 172.16.105.84:80
```

  Look at this output part: '[1:123:1]', the first component is `gid=1`, the second is `sid=123` and the last one is `rev=1`.

5. **classtype** (Classification Type)

   Provide the classification of the alert. This is used to categorize the rules and the alert.

   - **Syntax:** `classtype:<type>;`
   - **Example:** `classtype:attempted-admin;`

   When writing rules, if we assign it to a classification type using `classtype`, we must define that classification type first inside the `/var/lib/suricata/rules/classification.config` file. You can open this file to see the list of available classification types.

```
chutrunganh@chutrunganh:~$ sudo su
[sudo] password for chutrunganh:
root@chutrunganh:/home/chutrunganh# cd /var/lib/suricata/rules/
root@chutrunganh:/var/lib/suricata/rules# ls
classification.config  suricata.rules  test.rules
root@chutrunganh:/var/lib/suricata/rules# cat classification.config
config classification: not-suspicious,Not Suspicious Traffic,3
config classification: unknown,Unknown Traffic,3
config classification: bad-unknown,Potentially Bad Traffic, 2
config classification: attempted-recon,Attempted Information Leak,2
config classification: successful-recon-limited,Information Leak,2
config classification: successful-recon-largescale,Large Scale Information Leak,2
config classification: attempted-dos,Attempted Denial of Service,2
config classification: successful-dos,Denial of Service,2
config classification: attempted-user,Attempted User Privilege Gain,1
config classification: unsuccessful-user,Unsuccessful User Privilege Gain,1
config classification: successful-user,Successful User Privilege Gain,1
```

Figure 0.7: Class types inside config file

Each classification type follow this format:

`config classification: <short_name>,<full_name>,<priority_level>`

Example: `config classification: attempted-admin,Attempted Administrator Privilege Gain,1`

Once we have defined the classification in the configuration file, we can use the classtype in our rules. A rule with classtype, for example `classtype: attempted-admin` will be assigned a priority of 1 and the alert will contain `Attempted Administrator Privilege Gain` in the Suricata

logs.

6. **priority**

   Use to set the priority of the rule. The higher the number, the higher the priority. Highest is 1 and lowest is 255. Signature with higher priority will be processed first. Normally, if a rule already using `classtype`, then the priority filed is already included in the classtype definition, so you do not need to add `priority` in the rule unless you want to override the priority of the classtype.

   - **Syntax:** `priority:<number>`
   - **Example**: `priority: 1`

7. **reference**

   Provides a reference to an external source or document related to the rule that can provide more information about the rule. This `reference` keyword can appear multiple times in a rule.

   - **Syntax:** `reference:<type>,<reference_source` with `type` keyword can be `url`, `cve`, etc. See all reference types that are defined in your Suricata by checking the `reference.config` file under `/etc/suricata/`:

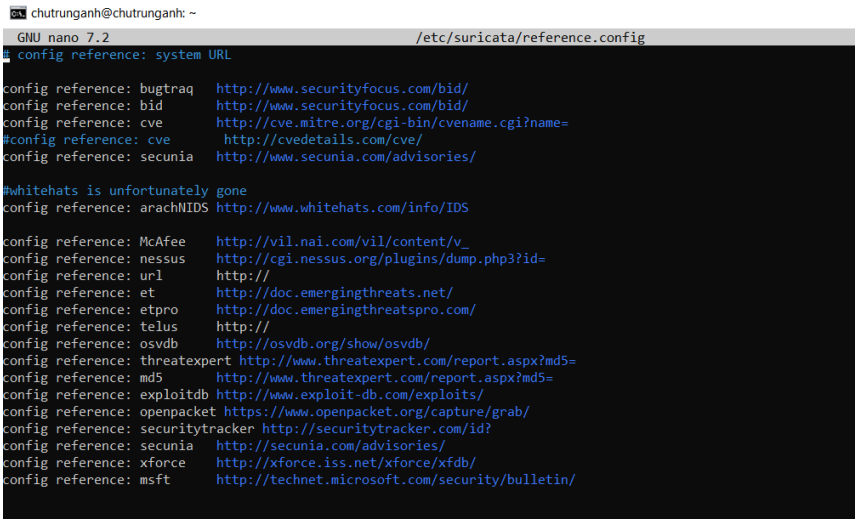


Figure 0.8: Reference types defined inside Suricata config file

   - **Example**: `reference:url,https://www.example.com/rule-info` or `reference:cve,CVE-2025-24985`

8. **metadata**

   Use to add additional information about the rule in free-form, but usually in

key-value pair since Suricata can include these in the eve formatted log, which is in JSON format. This is useful for adding information such as the author of the rule, the date the rule was created, or any other information that may be relevant to the rule

- **Syntax:** `metadata: <key 1> <value 1>, <key 2> <value 2>`
- **Example:** `metadata: author CTA, created_at 2025_04_21`

9. **target**

   Used by rule writer to indicate which side of the communication is the **target (victim)** of the attack. By default, Suricata assumes the destination IP is the target, but in some cases (like data exfiltration), the **source IP** may actually be the intended target. This keyword improves the accuracy of the generated alert by explicitly marking the target side. The resulting event in JSON (EVE format) will include a separate field for this.

   - **Syntax:** `target: [src_ip | dest_ip]`. If thr
   - **Example:** `target: dest_ip` indicate that the destination IP address machine is the target of the attack

10. **requires**

    Allows a rule to specify that certain Suricata features must be enabled or a certain Suricata version must be used. If the environment does not meet these requirements, the rule will be ignored. This is useful for rules that require specific features/ version to be enabled in order to work properly. For example, if a rule requires the `http_inspect` preprocessor to be enabled, you can use this option to specify that requirement.

    - **Syntax:** `required: feature <feature_name>, version <version_condition>`
    - **Example:** `required: feature geoip, version >= 7.0.4 < 8` . This means the rule will only be loaded if the `geoip` feature is enabled and the Suricata version is at least 7.0.4 but less than 8.

b, Payload Options

Inspect the payload content of a packet or a stream.

1. **content** (Content Match)

   If no sub option is added, try to find a match in all bytes of the payload

- **Syntax:** `content:"<string>";` this string can be in hex format or ASCII format. If you want to use the hex format, place it between `| |`
- **Example:** `content:"A|3A|b|"` which represents the string `A:b` literally.

Some sub-options that are used in conjunction with with `content` option:

- **`replace`**: The this options can only be used in IPS, to modify the packet payload. Example usage: `content:"abc"; replace: "def".` This will changes the content matching part found in packet payload from text (`'abc'`) to (`'def'`). This is useful for replacing sensitive information in network traffic.
- **`nocase:`** to make the match case insensitive. Example: `content:"A|3A|b"; nocase;` can match with `A:b`, `a:b`, `A:B`, `a:B`.
- **`depth:`** to limit the search to the first n bytes of the payload. Example usage: `content:"A|3A|b"; depth: 3;` will only search the first 3 bytes of the payload. Example string `A:bcdefghik` in payload will match since three first bytes `A:b` is same as the content.
- **`offset:`** to skip the first n bytes of the payload. Example: `content:"A|3A|b"; offset: 2;` will skip the first 2 bytes of the payload. Example string `CdA:b` in payload will match since two first bytes `Cd` are ignored.
- **`distance`**: to fine-tune how close to search for the next content match relative to a previous one. It's super useful for detecting patterns that occur at a specific offset after a previous match. Syntax:

```
content:"first_pattern";
content:"second_pattern"; distance:<number>;
```

Example in a rule:

```
alert tcp any any -> any 80 (
    msg:"Suspicious HTTP request with specific pattern";
    content:"GET"; http_method;
    content:"/admin"; distance:5; http_uri;
    sid:1000001; rev:1;
)
```

First, it looks for `"GET"` in the HTTP method. Then, starts scanning 5

bytes after the end of `"GET"` for `"/admin"` text in the URI. Here is a visualization:

```
[---"GET"---]--(5 bytes gap) --[--"/admin"--]
 ^ First match                  ^ Start second match here
```

In case `/admin` appears too soon (less than 5 Bytes gap, then it will not match. So `distance` keyword limits the "closest" gap allow to start searching for the next content match relative to a previous one. We can combine with **`within`** keyword to limit the "farthest" gap that still accept searching for the next content match relative to a previous one. This helps create a range of bytes to search for the next content match. Example:

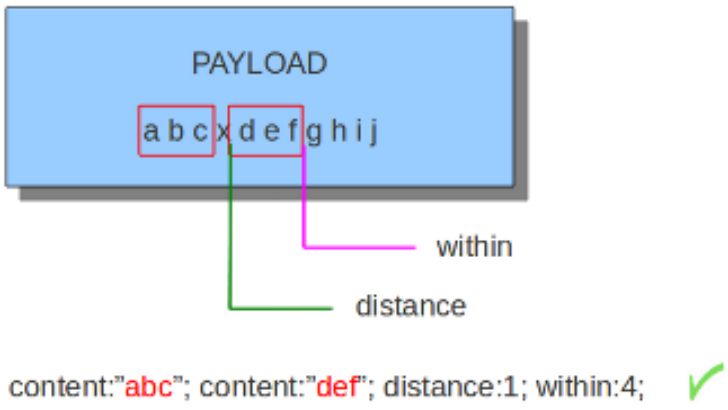


Figure 0.9: This rule will only match if the string `'def'` appears in a gap between 1 to 4 Bytes counting from the end of the string `'abc'`.

- **`isdataat`**: Verifies whether there is still a specified number of bytes existing at or after a certain offset (absolute or relative). Syntax: `isdataat:<number>` With the `relative` option is used to know if there is still data at a specific part of the payload relative to the last match. Examples:

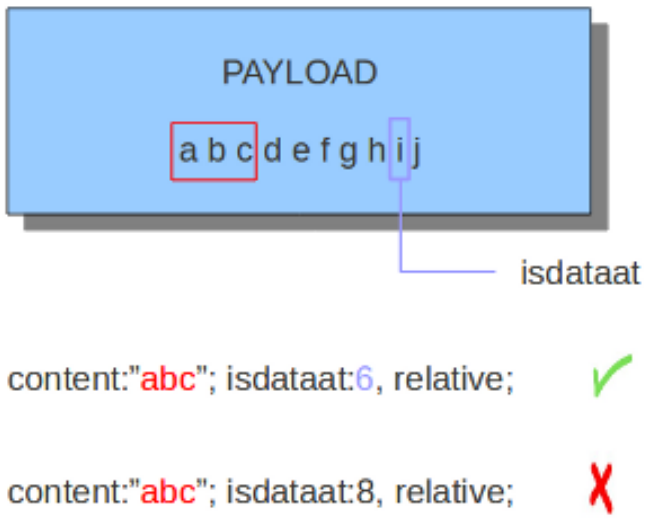


Figure 0.10: Start from the last mach, letter ”d” in this case, count till the end of payload, there are only 7 bytes left.

2. **`dsize`**

   Checks the payload size of the packet. It's useful for detecting unusually sizes of payloads, like in detecting buffer overflows or other attacks that rely on large payloads.

   - **Syntax:**

     ```
     dsize:<comparison><value>; or dsize:min<>max;
     ```

   - **Example**:

     ```
     alert tcp any any -> any any (msg:"dsize more than
     or equal value"; dsize:>=10; sid:3; rev:1;)
     ```

     This rule will be triggered if the payload size of package is more than 10 bytes. Another example:

     ```
     alert tcp any any -> any any (msg:"dsize range value";
     dsize:8<>20; sid:6; rev:1;)
     ```

     Triggers if the payload size is outside range 8 to 20 bytes.

3. **`entropy`**

   Calculate the Shannon entropy value of the content and compare it with a given value. This is useful for detecting encrypted or compressed data, which often has high entropy. The entropy value is calculated based on the randomness of the payload. High entropy values indicate that the data is more random and less predictable, which can be a sign of encryption or obfuscation.

   - **Syntax:**

     ```
     entropy:<operator><value>
     ```

     Range: 0.0 (very predictable) to 8.0 (very random).

   - **Example usage:**

     ```
     alert http any any -> any any (msg:"entropy simple
     test"; file.data; entropy: value >= 7.5; sid:1;)
     ```

     This example matches if the file.data content for an HTTP transaction has a Shannon entropy value higher than 7.5 (possibly encrypted or obfuscated).

4. **`pcre`** (Perl Compatible Regular Expressions)

   Allows matching of patterns using Perl-compatible regular expressions (PCRE). Because PCRE can be performance-intensive, it is typically used together with `content|`. In such cases, the `content` must match first, and only then is the PCRE evaluated — reducing unnecessary processing.

   - **Syntax:**

     ```
     pcre:"/<regex>/options";
     ```

     `options` can be: `i` for case insensitive, `s` for not checking newline characters, `E` for ignoring the newline characters at the of the buffer/payload, `R` for matching relative to the last pattern match (similar to distance 0), etc

   - **Example:** In this example following:

     ```
     pcre:"/[0-9]{6}/";
     ```

     There will be a match if the payload contains six numbers

There are still some more options keyword to work with packet payload: `startwith`, `endswith`, `absent`, `bsize`, `byte_test`, `byte_math`, `byte_jump`, `byte_extract`, `rpc` that have not been covered in this report. For detailed explanations of these keywords, refer to the Suricata payload keywords.

c, Non-payload Options

Non-payload options depend on various packet characteristics such as header fields, protocol types, and flow attributes. A detailed explanation is beyond the scope of this project.